阅流 RSS —— 本地优先的 RSS 阅读器

主 App（B3 RSS 阅读器）+ 伴生模块（P6 备份与恢复）

项目	内容
课程名称	《Android 移动终端应用开发技术》（2 学分）与《移动互联网应用开发实战》（1 学分）
项目题目	主 App：B3 RSS 阅读器；伴生模块：P6 备份与恢复
姓名	邬逸帆
学号	202405100113
班级	物联网 241
指导教师	邱景
完成日期	2026 年 11 月 13 日
Git 仓库	https://github.com/xu2hou/RSS.git

目录

1. 1 需求分析
2. 2 系统设计
3. 3 数据库设计
4. 4 接口设计
5. 5 关键功能实现
6. 6 伴生模块设计（P6 备份与恢复）
7. 7 界面设计
8. 8 测试
9. 9 总结与展望
10. 10 参考文献与附录

1 需求分析

1.1 项目背景与目标

RSS 是一种「订阅而不是被算法推荐」的信息获取方式。国内可用的 RSS 客户端要么已经停止维护，要么把阅读、订阅、缓存全部放在云端，离线时就完全不可用。本项目要做的是一个本地优先的阅读器：订阅与文章全部落库在手机本地，断网时仍能读已缓存的内容，联网时增量同步。

一句话定位：一个用原生 Android 实现、可离线阅读、能自动备份与恢复的 RSS 阅读器。

目标用户：有固定信息源（技术博客、行业媒体）并且希望在手机上集中阅读、不希望被推荐流干扰的人。

1.2 功能需求

模块	功能描述	优先级
订阅源管理	添加订阅（地址校验、抓取预览、推荐源）、列表展示未读数、长按取消订阅	高
文章列表	推荐 / 未读 / 收藏三档筛选，已读置灰，星标收藏	高
文章阅读	WebView 渲染正文、自适应排版、深色模式、分享、浏览器打开原文	高
后台刷新	WorkManager 每 6 小时联网刷新，失败按健康度退避	高
新文章提醒	发现新文章时通知；无通知权限时降级为仅应用内提示	中
推荐排序	按收藏、未读、订阅源健康度加权并对时间衰减，本地计算	中
设置	自动刷新开关、通知授权入口、后台任务状态、伴生模块入口	中
备份与恢复（伴生）	一键备份、导出/导入 JSON、合并式恢复、每日自动备份	高

1.3 非功能需求

- 性能：列表滚动不卡顿（RecyclerView + DiffUtil 局部刷新）；抓取与入库全部在后台线程，主线程不做 I/O。

- 离线优先：文章正文与阅读状态本地存储，断网仍可阅读已缓存文章。
- 稳定性：单条源抓取失败不影响整轮刷新；异常统一翻译为中文可读文案，不把堆栈抛给用户。
- 兼容性：minSdk 24、targetSdk 35，Android 15 强制边到边显示需要显式处理窗口内边距。
- 隐私：不做任何埋点与上报；备份文件默认写在应用私有目录，导出才由用户主动选择位置。

1.4 用户与使用场景

典型用户：学生 / 开发者，每天通勤时想读几个固定的技术源。

- 场景一：早上打开 App，下拉刷新，看到「刷新完成：成功 3 个，新增 7 篇」，在推荐档读完两篇。
- 场景二：在地铁里没有信号，打开昨天缓存过的文章继续读，阅读状态在本地记录，出站后自动同步。
- 场景三：换手机前打开「备份与恢复」做一次备份，导出到网盘；新机上导入，订阅与阅读状态都还在。

2 系统设计

2.1 总体架构

采用 MVVM 分层，自下而上四层：

```
UI 层     Activity / Fragment (ViewBinding)
              ↓ 只观察 LiveData、只调用 ViewModel 方法
ViewModel     AndroidViewModel + LiveData；持有 Repository，不持有 View
              ↓
Repository    FeedRepository / ArticleRepository：编排网络、解析、事务入库
              ↓
数据源      Room (AppDatabase) / OkHttp (RssFetcher) / SharedPreferences
```

异步模型：Java 项目不使用 Kotlin 协程，改用「ViewModel + LiveData + ExecutorService」实现等价的「后台计算、主线程渲染」：单线程 diskIO 保证数据库写入顺序，三线程 networkIO 允许并发抓取，结果统一通过 Handler 切回主线程。

2.2 模块划分

包 / 模块	职责	关键类
data.entity	Room 实体与投影	Feed、Article、FeedWithUnread
data.dao	数据访问（含带未读数的联表查询）	FeedDao、ArticleDao
data.net	抓取与解析、错误文案映射	RssFetcher、RssParser、NetworkErrorMapper
data.repo	业务编排：抓取 → 解析 → 事务入库	FeedRepository、ArticleRepository
logic	自定义业务逻辑（纯函数、可单测）	ArticleRanker、FeedHealth
work	后台定时刷新	RefreshWorker、RefreshScheduler
notify	通知与权限降级	NotificationHelper
ui.*	界面层	MainActivity、FeedsFragment、ArticlesFragment、ArticleDetailActivity
backup	伴生模块核心	BackupPayload、BackupManager、AutoBackupWorker
ui.backup	伴生模块界面（独立入口）	BackupActivity、BackupViewModel

2.3 技术选型

技术 / 库	用途	选型理由
Java 17 + XML View	语言与界面	课程要求原生 Android；Compose 仅支持 Kotlin，故用 View 体系
AndroidX AppCompat / Material 3	控件与主题	官方组件，DayNight 主题自带深色适配
Room 2.6.1	本地数据库	编译期校验 SQL、支持 LiveData 响应式查询、事务 API 清晰
OkHttp 4.12.0	HTTP 抓取	超时/重定向/拦截器配置成熟，比 HttpURLConnection 可控
XmlPullParser（系统内置）	XML 解析	流式解析、内存占用低，不额外

		引第三方 XML 库
WorkManager 2.9.1	后台任务	唯一任务名 + 约束，系统会保证执行，比 AlarmManager 省电
ViewModel + LiveData	状态与生命周期	屏幕旋转不丢状态，观察者随生命周期自动解绑
ExecutorService	并发	Java 下替代协程的标准做法，线程模型显式可控
org.json（系统内置）	备份序列化	备份格式要长期可读，纯 JSON 最稳，且零额外依赖

2.4 数据流与关键流程

以「添加订阅」为例，数据依次流经：

```
AddFeedActivity 输入地址
  → AddFeedViewModel.addFeed(url)
    → FeedRepository.addFeed()            [networkIO 线程]
      → RssFetcher.normalizeUrl()         校验与规范化
      → RssFetcher.fetch()                OkHttp 取字节流（限 8MB）
      → RssParser.parse()                 RSS 2.0 / Atom 双格式解析
      → database.runInTransaction()       插 feed + 批量插 articles  [diskIO 线程]
    ← Callback.onSuccess / onError
  ← LiveData 事件 → 主线程 Toast / 错误文案
```

「列表刷新」不走回调，而是靠 Room 的响应式查询：任何写入都会让 observeAllWithUnread() 重新发射，界面自动更新，不需要手动通知。

3 数据库设计

3.1 ER 关系

两张表，一对多关系：一个订阅源（feeds）对应多篇文章（articles），外键 articles.feedId → feeds.id，删除源时级联删除其文章。

```
feeds  1 ──────── n  articles
  id (PK)              id (PK)
  url (UNIQUE)         feedId (FK → feeds.id, CASCADE)
  title                link (UNIQUE)
  failCount            publishedAt (INDEX)
  ...                  read / starred ...
```

3.2 表结构

表名：feeds

字段	类型	约束	说明
id	INTEGER	PK, AUTOINCREMENT	主键
title	TEXT	NOT NULL	源标题，解析失败时退化为域名
url	TEXT	NOT NULL, UNIQUE	订阅地址，去重依据
siteUrl	TEXT		站点主页
description	TEXT		源描述（截断 120 字）
lastFetchedAt	INTEGER		最近一次成功抓取时间
failCount	INTEGER		连续失败次数，用于健康度与退避
enabled	INTEGER		是否参与后台刷新
createdAt	INTEGER		订阅创建时间

表名：articles

字段	类型	约束	说明
id	INTEGER	PK, AUTOINCREMENT	主键
feedId	INTEGER	FK → feeds.id, ON DELETE CASCADE, INDEX	所属订阅源
title	TEXT	NOT NULL	文章标题
link	TEXT	NOT NULL, UNIQUE	原文链接，去重依据
author	TEXT		作者
summary	TEXT		纯文本摘要（列表展示）
contentHtml	TEXT		正文 HTML（详情页 WebView 渲染）
publishedAt	INTEGER	INDEX	发布时间戳，列表排序依据
read	INTEGER		是否已读
starred	INTEGER		是否收藏
fetchedAt	INTEGER		入库时间

3.3 索引与约束的取舍

- feeds.url 唯一索引：从数据库层面防止重复订阅，避免只靠 Java 判断产生竞态。
- articles.link 唯一索引：同一次刷新、多次刷新都不会产生重复文章（配合 OnConflictStrategy.IGNORE）。
- articles.publishedAt 加索引：三档列表都按发布时间倒序，扫描最频繁。
- 外键级联：删除订阅源时自动清理文章，避免出现孤儿记录。

4 接口设计

4.1 网络接口清单

本项目不依赖自有后端，直接抓取第三方 RSS / Atom 源，因此「接口」体现为抓取约定：

编号	方法	URL	请求头	响应	说明
API-1	GET	订阅源地址（用户填写，如 https://sspai.com/feed）	User-Agent: YueLiuRss/1.0 Accept: application/rss+xml, application/atom+xml, application/xml	RSS 2.0 或 Atom 的 XML	通用抓取；跟随重定向，超时 12/20/30 秒
API-2	GET	任意 http(s) 地址	同上	可能返回非 XML 内容	用于识别「不是有效 RSS」，此时抛业务异常而不是解析崩溃

已实测可用：少数派（RSS 2.0）、Solidot（RSS 2.0）、IT 之家（RSS 2.0）、阮一峰的网络日志（Atom）。实测不可用：V2EX（超时）、36 氪（返回 HTML）、BBC（TLS 失败），演示时已避开。

4.2 错误码与异常约定

不向外暴露原始异常对象，统一由 NetworkErrorMapper 翻译成「用户能据此行动」的中文：

异常类型	用户看到的文案	处理策略
UnknownHostException / NoRouteToHostException	网络不可用：无法解析服务器地址，请检查网络连接	列表非空时 Snackbar + 重试；列表为空时铺满错误态
ConnectException	网络不可用：连接服务器失败，请检查网络	同上
SocketTimeoutException	连接超时：服务器响应太慢，请稍后重试	同上
SSLException / CertificateException	安全连接失败：证书校验未通过或网络被拦截	同上
XmlPullParserException	解析失败：该地址返回的内容不是有效的 RSS / Atom	添加订阅页回显在输入框下方
HTTP 非 2xx	服务器返回 HTTP 4xx / 5xx	计入 failCount，触发退避
响应体 > 8MB	源文件过大（>8MB），已跳过	直接跳过，不下载完

5 关键功能实现

5.1 RSS 2.0 与 Atom 双格式解析

两种格式的结构差异是本节核心：RSS 的链接是标签文本 <link>url</link>，Atom 的链接是属性 <link rel="alternate" href="url"/>；RSS 正文常用 <description>，Atom 用 <content>。解析器用 XmlPullParser 流式读取，以「有没有 href 属性」区分两种形态：

```
case START_TAG:
    if ("link".equals(name)) {
        String href = parser.getAttributeValue(null, "href");
        if (href != null) {            // Atom 形态
            currentItem.link = href.trim();
        } else {                       // RSS 形态：等标签文本
            currentTag = name;
        }
    }
```

另外三个工程细节：

- 关闭命名空间处理，按「标签名 + 前缀」直接匹配，从而兼容 content:encoded、dc:date 这类带前缀的标签。

- encoder 未知时传 setInput(stream, null)，由解析器按 BOM 与 <?xml encoding?> 自行判定编码，UTF-8 与 GBK 源都能读。
- 单条 item 解析失败不影响其它条目；没有 link 的条目直接丢弃（既无法去重也无法打开）。

该解析器被抽成两层：外层 parse(InputStream) 用系统 Xml，内层 parse(XmlPullParser, InputStream) 允许注入解析器实现 —— 因此可以在普通 JVM 单元测试里用 kxml2 跑回归，不必启动模拟器。

5.2 自定义推荐排序算法

「推荐」档不是简单的按时间倒序，而是一个可解释的加权评分：

```
raw   = 1.0                          基础分
      + 3.0  若已收藏
      + 1.0  若未读
      + 2.0 × health                 源健康度 ∈ [0,1]
score = raw / (ageHours / 6 + 1) ^ 1.2
```

时间项让新文章有机会压过「老的高分文章」，指数 1.2 控制衰减速度，6 小时是半衰参考尺。实现为纯函数 ArticleRanker.score()，不依赖 Android，单元测试可直接断言数值关系。

5.3 订阅源健康度与失败退避

一个长期失效的源不应该拖垮每次刷新。连续失败按指数退避，未到期的源在后台刷新时被跳过：

连续失败次数	下次可重试间隔	界面提示
0	立即可抓	正常
1	10 分钟	失败 1 次，N 分钟后重试
2	1 小时	失败 2 次，N 小时后重试
3	6 小时	异常，约 N 小时后重试
≥4	24 小时	异常

健康度 healthScore 同时作为排序权重：0 次失败 = 1.0，失败 1 次 = 0.67，达到阈值 3 次 = 0。也就是说「经常抓失败的源」在推荐流里自然降权。

5.4 后台刷新与通知（含权限降级）

WorkManager 登记唯一周期任务，周期 6 小时并约束「联网」；Worker 在后台线程同步执行抓取入库，发现新文章且用户开启提醒时发通知。任务状态在设置页显式展示：

```
后台任务状态：ENQUEUED（周期 6 小时）
```

权限降级是本节重点。Android 13+ 的 POST_NOTIFICATIONS 属于运行时权限，用户拒绝后：

- 应用不崩溃、不静默失败：刷新与阅读功能完全不受影响；
- 设置页如实显示「已拒绝通知权限。功能仍可正常使用，需要提醒时点下方按钮重新授权」；
- NotificationHelper 在发通知前检查权限，并在 notify 外层捕获 SecurityException，避免权限在调用瞬间被撤销导致后台任务整体失败。

5.5 难点与解决方案

难点	现象	解决思路	结果
Android 15 强制边到边	targetSdk 35 后，列表内容顶到状态栏下面，未读徽标与系统时间重叠	用 ViewCompat.setOnApplyWindowInsetsListener 按 systemBars 给根布局加内边距；fitsSystemWindows 在强制边到边下不管用	所有页面内容正确避让，截图不再压状态栏
AGP 8 默认不生成 BuildConfig	设置页显示版本号时编译报「找不到符号 BuildConfig」	显式开启 buildFeatures.buildConfig = true	版本号可正常读取
WorkManager 数据库被写坏	JobScheduler 里任务还在，但 workdb 变成 0 字节，定时任务静默不执行	设置页增加「后台任务状态」实时展示 ENQUEUED，异常时能立刻发现；重新登记任务即可恢复	从「界面毫无感知」变成「一眼可见」
Gradle 构建缓存损坏	异常中断后构建报 Invalid classfile header / invalid stream header	删除本地 build-cache-1 并 --no-build-cache 重建	构建恢复，APK 正常产出

6 伴生模块设计（P6 备份与恢复）

6.1 模块定位与主 App 的关系

伴生模块是一个独立可启动的单元：它在桌面上有自己的图标（第二个 LAUNCHER 入口），也能从主 App 设置页或长按图标的应用快捷方式进入。它不做「另一个 App」，只做一件事：把主 App 的订阅与阅读状态完整地备份出去、再恢复回来。

6.2 实现要点

- 备份格式：纯 JSON，带 schemaVersion 版本号、exportedAt 时间与 appVersion；feed 与 article 分开存放。
- 导出：默认写应用私有目录 files/backups/（自动命名为 yueliu-backup-yyyyMMdd-HHmmss.json），也可以通过 SAF（ACTION_CREATE_DOCUMENT）导出到用户选择的位置。
- 自动备份：WorkManager 每日一次，写入后只保留最近 7 份，超出部分自动清理。
- 记录列表：展示文件名、时间、大小；点击可恢复，右侧菜单支持分享（FileProvider）与删除。
- 异常处理：文件为空、非 JSON、缺少 schemaVersion、版本过新、条目缺少关键字段，都给出明确中文提示，不崩溃。

6.3 与主 App 的数据打通

数据打通方式的选择是本节的设计要点：

方案	做法	结论
复制 .db 文件	直接把 databases/yueliu.db 拷出来	不采用：WAL 模式下拷到的主文件可能不含最新事务，快照不一致
跨进程 ContentProvider	把 Room 包装成 Provider 供别的 APK 访问	不采用：本项目同进程，增加复杂度却无收益
复用同一个 Room 实例（采用）	BackupManager 直接注入 AppDatabase，用 DAO 读全量、事务写回	采用：快照一致、事务安全、代码最简

分享备份文件走 FileProvider（authorities = ${applicationId}.fileprovider），路径白名单只开放 files/backups/，不暴露整个应用私有目录。

6.4 恢复的合并策略

数据	去重键	已存在时	不存在时
订阅源	url	复用本机已有的 id，跳过新增	新建记录，并记录「备份 id → 本机 id」映射
文章	link	按用户选择：覆盖 read/starred，或保留本机状态	以映射后的 feedId 新建
源已不存在的文章	—	跳过并计数	跳过并计数

整个恢复在一个 Room 事务里完成：要么全部成功，要么回滚，不会留下「导入一半」的脏数据。

7 界面设计

7.1 页面清单

页面	功能	对应界面
订阅列表	订阅源列表、未读数、下拉刷新、长按删除、空态	图 7-1
添加订阅	URL 输入与校验、推荐源快捷填入、错误回显	（见录屏）
文章列表	推荐 / 未读 / 收藏三档、已读置灰、星标	图 7-2
文章详情	WebView 正文、深色模式、收藏/分享/浏览器	图 7-3、图 7-4
设置	自动刷新、通知授权、后台任务状态、伴生模块入口	（见录屏）
备份与恢复（伴生）	立即备份、导出/导入、每日自动备份、记录列表	图 7-5



图 7-1 订阅列表与未读徽标


推荐
未读
收藏
派早报：国务院办公厅发布 2027 年部分节假日放假安排等
少数派 · 11-06
OpenAI 推出 Images 2.6 模型，Nothing CMF Speaker S/M 音箱国行上市等。 查看全文
开启 Mac 的下一世代：MacBook Pro (PROLED XDR) 首发体验
少数派 · 11-05
全新 MacBook Pro，开启 Mac 的下一个世代。 查看全文
新玩意 255 | 少数派的编辑们最近买了啥?
少数派 · 11-05
编注：很多读者都会好奇少数派的编辑们到底平时都「买了啥」。我们希望通过「编辑部的新玩意」介绍编辑部成员们...
不用困在厨房也能做出好饭：三十道省事家常蒸菜
少数派 · 11-05
蒸锅真是懒人做饭好帮手。 查看全文 本文为会员文章，出自《单篇文章》，订阅后可阅读全文。
Apple 官网排版细节鉴赏
少数派 · 11-05
Apple 独一无二的品牌价值，正是从这些细节中凝聚的。 查看全文
派早报：腾讯发布混元 Hy4 正式版，Google 调整 K-12 教育 AI 政策等
订阅
文章
设置

图 7-2 文章列表「推荐」档



图 7-3 文章详情（WebView 渲染正文，图片自适应）



图 7-4 详情页深色模式（CSS prefers-color-scheme）



图 7-5 伴生模块：备份与恢复

7.2 交互与状态说明

加载态 / 空态 / 错误态统一封装为 StateView 控件，任何列表页两行代码就能补齐三态，避免每个 Fragment 各写一套显隐逻辑：

- 加载态：居中进度条 + 文案，用于抓取解析过程中。
- 空态：按档位给不同文案（如「没有未读文章，去「推荐」看看」），而不是一句「暂无数据」。
- 错误态：显示具体原因 +「重试」按钮；列表已有内容时改用带「重试」动作的 Snackbar，不打断阅读。



图 7-6 断网刷新失败：Snackbar 给出原因与重试

8 测试

8.1 测试环境

项	内容
设备	Android Emulator，AVD Pixel_API35（1080×2400，Android 15 / API 35）
加速	AEHD 2.2（无需 Hyper-V，模拟器全速运行）
构建	Gradle 8.9 + AGP 8.7.3 + JDK 21，compileSdk 35 / minSdk 24
单元测试	JUnit 4.13.2 + kxml2 2.3.0（JVM 上替代

	Android 的 Xml）+ org.json 实现

8.2 单元测试

共 30 个用例，全部通过（gradlew testDebugUnitTest）：

测试类	用例数	覆盖内容
DateUtilsTest	5	RFC 822 / ISO 8601 解析、非法输入返回 0、相对时间文案
HtmlUtilsTest	5	去标签、去 script/style、实体反转义、截断、null 安全
RssParserTest	4	RSS 2.0 频道与条目、时间解析、Atom 的 href 链接与 CDATA 正文、非 RSS 内容
ArticleRankerTest	6	新鲜度、收藏加权、健康度降权、排序与条数限制、空输入、缺失发布时间兜底
FeedHealthTest	5	退避梯度、退避窗口判定、健康度衰减、异常阈值、文案
BackupPayloadTest	5	JSON 往返字段一致、版本过新拒绝、非 JSON 拒绝、缺 schemaVersion 拒绝、脏记录过滤

8.3 手工测试用例与结果

编号	测试项	步骤	预期	结论
T-1	添加订阅	添加订阅页选「少数派」→ 添加	列表出现该源，未读徽标 10	通过
T-2	双格式解析	再添加「阮一峰的网络日志」（Atom）	列表出现该源，未读徽标 3	通过
T-3	断网刷新	飞行模式下下拉刷新	Snackbar 提示「网络不可用…」并带重试	通过
T-4	三档筛选	切换 推荐 / 未读 / 收藏	筛选结果正确，空档显示对应文案	通过
T-5	星标联动	详情页点星标 →	该文章出现在收	通过

		返回 → 收藏档	藏档	
T-6	已读联动	打开文章 → 返回 → 未读档	该文章从未读档消失	通过
T-7	深色模式	cmd uimode night yes 后打开文章	正文背景 #121417、文字转浅色	通过
T-8	通知权限拒绝	设置页点「开启通知权限」→ Don't allow	页面提示已拒绝，刷新仍正常	通过
T-9	新文章通知	授权后强制触发 Worker（有新文章）	通知栏出现「后台刷新发现 7 篇新文章」	通过
T-10	备份	备份页点「立即备份到本机」	生成 json，记录列表出现该文件	通过
T-11	恢复	清空 feeds 后从备份恢复	订阅源 0→3、文章 87、收藏状态保留	通过
T-12	独立入口	am start BackupActivity（等同点桌面图标）	可直接拉起备份页	通过

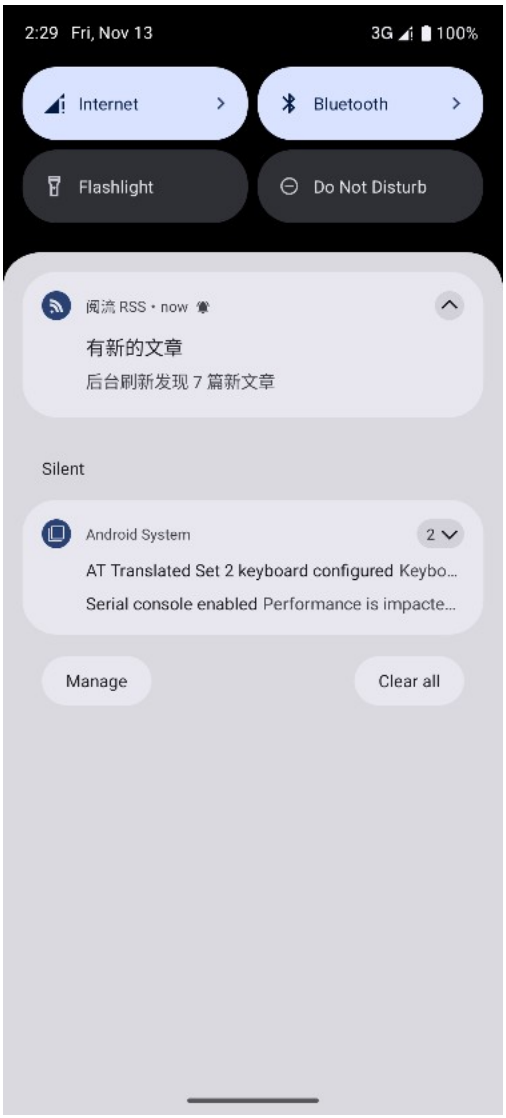


图 8-1 新文章通知（后台刷新发现 7 篇）

8.4 问题记录与修复

问题	定位过程	修复提交
状态栏遮挡内容	第一版截图发现未读徽标与系统时间重叠，结合 targetSdk 35 判断为强制边到边	fix: 修复 Android 15 边到边导致状态栏遮挡内容
BuildConfig 找不到	编译报「找不到符号: 类 BuildConfig」，查 AGP 8 变更说明确认默认关闭	feat: 搭建应用骨架与底部导航（含 BuildConfig 开关）
网络异常显示英文	断网实测 Snackbar 出现 Unable to resolve host	feat: 网络异常映射为可读中文
WorkManager 静默失效	强制触发 job 后无任何 Worker	feat: 设置页开关、任务状态展

	日志，查 databases 发现 workdb 为 0 字节	示
两个单测断言写错	30 个用例中 2 个失败，逐个看 test-report 确认是断言假设错误而非产品缺陷	test: 修正解析器与排序的断言假设
构建缓存损坏	异常断电后报 Invalid classfile header，删除 build-cache 后恢复	（环境问题，未改代码）

9 总结与展望

完成情况：主 App 五个界面（订阅、添加、文章、详情、设置）全部可用，覆盖「列表 → 详情」导航；Room 持久化 + 真实网络抓取 + WorkManager 后台任务三项硬能力齐备；权限被拒绝后有明确降级；自定义的推荐排序与健康度退避是真正在跑的业务逻辑，不是装饰。伴生模块独立可启动、能读写主 App 数据、有自动备份与滚动清理，异常路径都有可读提示。

收获：

- 把「异步与生命周期」真正做对了：Repository 不碰 UI、ViewModel 不碰 View、所有 I/O 有明确线程归属，屏幕旋转后状态不丢。
- 学会用系统机制而不是猜：WorkManager 的 job 是否真的登记，看 dumpsys 而不是看代码；窗口是否真的避让，看截图而不是看布局文件。
- 理解了「可解释的工程」：自定义算法写成纯函数才能被单测覆盖，异常映射写成一层才能被复用。

不足与改进方向：

- 尚未支持 OPML 批量导入导出（RSS 客户端的通用标准），后续可加。
- 正文目前直接渲染源方 HTML，没有做阅读模式（去掉导航与广告），可引入可读性算法做二次净化。
- 备份为全量快照，数据量大时文件偏大（当前 87 篇文章约 340 KB），可改为增量或压缩。
- 尚未做多设备云同步与冲突合并，目前只做本地备份与合并式恢复。

10 参考文献与附录

参考文献

11. 1. Android Developers. Save data in a local database using Room. https://developer.android.com/training/data-storage/room
12. 2. Android Developers. Schedule work with WorkManager. https://developer.android.com/topic/libraries/architecture/workmanager
13. 3. Android Developers. Edge-to-edge enforcement for apps targeting SDK 35. https://developer.android.com/about/versions/15/behavior-changes-15
14. 4. RSS Advisory Board. Really Simple Syndication (RSS 2.0) Specification. https://www.rssboard.org/rss-specification
15. 5. RFC 4287. The Atom Syndication Format. https://www.rfc-editor.org/rfc/rfc4287
16. 6. Square Inc. OkHttp Documentation. https://square.github.io/okhttp/

附录 A Git 仓库与提交

仓库地址：https://github.com/xu2hou/RSS.git（分支 main）

提交信息统一为「<type>: <中文说明>」格式（feat / fix / test / docs / chore），按功能切片提交，覆盖从工程初始化到文档交付的全过程。提交作者为本人 GitHub 账号 XuZhou（与中文姓名邬逸帆为同一人）。

附录 B 运行说明

```
环境要求：JDK 17+（实测 21）、Android SDK platform 35、Gradle 8.9
构建：  ./gradlew assembleDebug
安装：  adb install -r app/build/outputs/apk/debug/app-debug.apk
单测：  ./gradlew testDebugUnitTest
```

注：仓库内 settings.gradle 已配置 Maven 阿里云镜像、wrapper 指向腾讯镜像，避免国内网络下拉取 Google Maven 长时间卡住。

附录 C 演示要点

- 先展示空态 → 添加订阅（推荐源一键填入）→ 列表出现未读数。
- 刷新一次，展示「刷新完成：成功 N 个，新增 M 篇」。
- 文章列表三档切换，点进详情看排版，切系统深色模式再展示一次。
- 设置页展示后台任务状态与通知授权；飞航模式演示错误态与重试。
- 最后进伴生模块：立即备份 → 记录列表 → 恢复并展示结果统计。